

TECHNICAL REPORT: SECURITY ARCHITECTURE ANALYSIS



UNIVERSIDAD
Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 12 - Security Architecture

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-04-19

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Introduction

1.1 Objective

The primary objective of this laboratory is to identify structural weaknesses in network architectures and propose improvements based on secure design principles. We aim to validate network exposures, implement robust firewall rules using modern Linux tools (`nftables`), and design segmented environments that adhere to the "Defense in Depth" strategy.

1.2 Discussion: The Architect's Perspective

Perspective: Andersson David Sánchez Méndez

"Security is not a product, but a process embedded in the architecture. A flat network is an invitation to lateral movement; by applying the principle of least privilege at the routing and filtering level, we ensure that a single compromise does not lead to a total system breach."

Perspective: Cristian Santiago Pedraza Rodríguez

"Resilience is built through layers. Understanding the difference between stateful inspection and application-layer filtering allows us to move beyond simple blocking toward an intelligent architecture that fails safely and maintains continuous visibility."



2. Core Principles of Secure Architecture

Our methodology is governed by six fundamental pillars that ensure a robust defensive posture:

Principle	Description	Implementation Strategy
Least Privilege	Minimal permissions for users/processes.	Strict ACLs and RBAC models.
Separation of Privileges	Dividing critical functions.	Multi-factor authorization for sensitive zones.
Minimal Exposure	Reducing the attack surface.	Disabling unused ports and services.
Defense in Depth	Multiple independent security layers.	Firewalls + IDS/IPS + Endpoint Protection.
Fail-safe Defaults	Default Deny posture.	Explicit 'Allow' rules, implicit 'Deny' at the end.
Continuous Auditing	Real-time monitoring and logging.	Centralized SIEM and syslog management.



3. Firewall Technologies & Architectures

A critical component of our architecture is the **Firewall**, acting as the primary gatekeeper for network traffic.

3.1 Firewall Classification

We categorize filtering technologies based on their inspection depth:

1. **Stateless (Packet Filtering)**: Inspects individual packets (IP/Port) without context.
2. **Stateful Inspection**: Tracks the state of active connections (`ct state established,related`).
3. **Next-Generation (NGFW)**: Integrates Deep Packet Inspection (DPI) and Layer 7 awareness (WAF).

3.2 Architectural Models

To protect the internal perimeter, we utilize established structural designs:

- ◆ **Bastion Host**: A single, hardened contact point for external services.
- ◆ **DMZ (Screened Subnet)**: Isolated zone for public-facing servers (Web, Mail).
- ◆ **Dual-Homed Host**: A gateway system with two interfaces separating distinct security zones.



4. Linux Firewalling: `nftables` Framework

In this laboratory, we transition from legacy `iptables` to `nftables`, the modern standard for Linux packet filtering. This tool provides a unified syntax and better performance for handling complex rule sets.

Key Syntax Example:

```
nft add rule inet filter input tcp dport 22 ct state new accept
```



5. PHASE 1: LABORATORY SETUP & NETWORK PROVISIONING

To validate secure architecture principles, we implemented a virtualized multi-segment environment using **VMware Workstation/Player**. This setup simulates a **Dual-Homed Host** architecture, where a single gateway (VM1) manages traffic between an untrusted external segment and a restricted internal segment.

5.1 Virtual Infrastructure & Custom VMnet Segmentation

We utilized **Custom VMnet Adapters** in VMware to create two strictly isolated broadcast domains. This ensures that traffic between the untrusted and trusted zones can only occur if the Gateway (VM1) explicitly permits it.

- ◆ **VMnet2 (External Segment):** Connects the Attacker (VM3) and the Firewall's external interface (`eth0`).
- ◆ **VMnet3 (Internal Segment):** Connects the Web Server (VM2) and the Firewall's internal interface (`eth1`).

VM	Role	Interface	Assigned IP	VMware Network
VM1	Firewall / Gateway	eth0 (Ext)	10.10.0.1/24	Custom (VMnet2)
		eth1 (Int)	10.20.0.1/24	Custom (VMnet3)
VM2	Web Server	eth0	10.20.0.10/24	Custom (VMnet3)
VM3	Attacker node	eth0	10.10.0.10/24	Custom (VMnet2)

5.2 Interface Provisioning & Routing

Static addressing was applied to avoid the use of DHCP, which can be a vector for network attacks and introduces non-deterministic variables. We manually defined the default gateways on the endpoints to force all inter-segment traffic through the security inspection point (VM1).

VM1 - Gateway Configuration:

Bash

```
# Activation of the dual-homed interfaces
sudo ip addr add 10.10.0.1/24 dev eth0 && sudo ip link set eth0 up
sudo ip addr add 10.20.0.1/24 dev eth1 && sudo ip link set eth1 up
```

VM2 & VM3 - Endpoint Routing:

Bash

```
# VM2 (Internal) pointing to the internal firewall interface
sudo ip route add default via 10.20.0.1

# VM3 (External) pointing to the external firewall interface
sudo ip route add default via 10.10.0.1
```



6. PHASE 2: BASELINE CONNECTIVITY & ISOLATION AUDIT

Before implementing security policies, we conducted a reachability audit to confirm the underlying network fabric and verify the **Minimal Exposure** posture.

6.1 Direct Adjacency Validation

Each endpoint was tested for connectivity against its local gateway interface.

- ◆ **External Segment:** VM3 (10.10.0.10) to VM1 (10.10.0.1) → **INITIAL FAILURE** (See Troubleshooting 6.3)
- ◆ **Internal Segment:** VM2 (10.20.0.10) to VM1 (10.20.0.1) → **SUCCESS**

6.2 Forwarding State & Segment Isolation

The Linux kernel's default state is to act as a host, not a router. We verified that **IP Forwarding** was disabled.

- ◆ **Forwarding Verification (VM1):** `cat /proc/sys/net/ipv4/ip_forward` → **Result: 0**
- ◆ **Lateral Movement Attempt:** `ping 10.20.0.10` from VM3 → **TIMEOUT**



6.3 Troubleshooting: Architectural and Connectivity Failures

During the initial provisioning, several structural issues prevented baseline communication. We performed a diagnostic process to align the virtual hardware with the logical configuration.

A. Hypervisor Network Mode Mismatch (NAT Conflict)

Initial reachability tests from VM3 failed with `Destination Host Unreachable`. Upon auditing the VMware settings, it was identified that the NICs were defaulting to **NAT** mode instead of **Custom (VMnet)**.

- ◆ **Impact:** The nodes were assigned to the hypervisor's management network (192.168.x.x) via DHCP, breaking the laboratory's static addressing and isolation.
- ◆ **Resolution:** All adapters were reconfigured to **Custom: Specific Virtual Network** (VMnet2 for external, VMnet3 for internal).

B. Logical-to-Physical Interface Mapping

A critical failure was identified using `ip neigh`. The gateway (VM1) showed an **ARP FAILED** state for VM3.

- ◆ **Root Cause:** The Linux kernel's interface enumeration (`eth0` , `eth1`) did not match the physical order of VMware's adapters. VM1 was attempting to reach the external segment through the internal wire.
- ◆ **Resolution:** A MAC-to-Interface audit was performed. Interfaces were flushed using `ip addr flush` and re-bound to the correct logical segments based on their hardware MAC addresses.

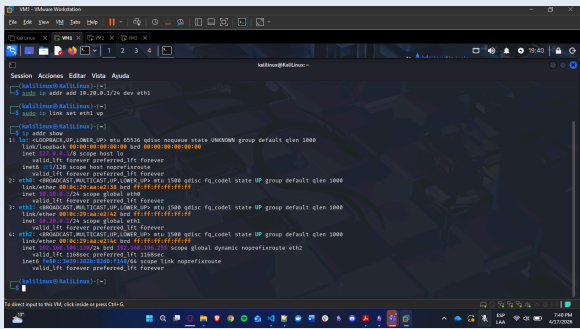
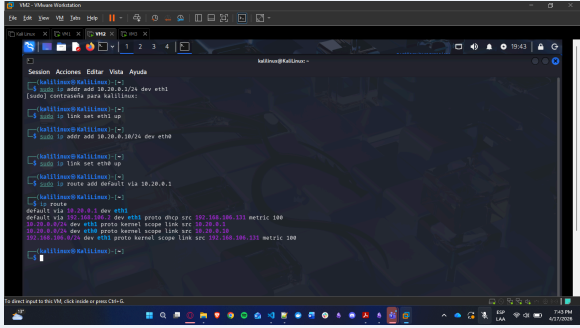
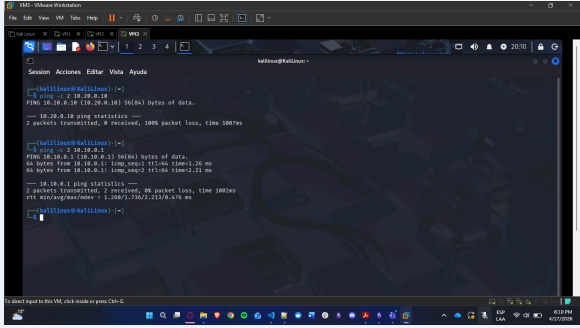
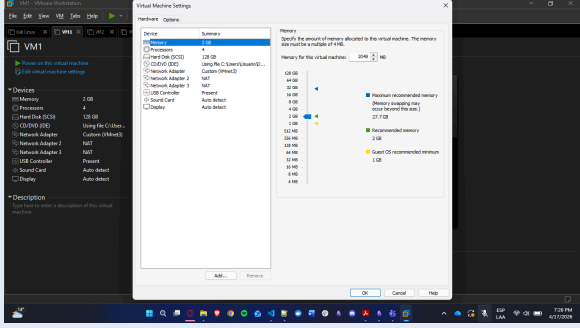
C. Network Manager Interference

Despite correct addressing, manual configurations were being overwritten.

- ◆ **Observation:** The `NetworkManager` service was attempting to reclaim control of the interfaces, causing "flapping" states.
- ◆ **Resolution:** The service was disabled (`systemctl stop NetworkManager`) to enforce strict static control over the security architecture.



Evidence Annex: Network Provisioning (VMware Edition)

Activity	Screenshot / Evidence	Purpose
VM1 IP Status		Show both <code>eth0</code> and <code>eth1</code> with their respective 10.10.0.1 and 10.20.0.1 IPs.
VM2 Default Route		Verify the line <code>default via 10.20.0.1</code> .
Forwarding Check	<code>sysctl net.ipv4.ip_forward</code>	Show the value <code>net.ipv4.ip_forward = 0</code> .
Isolation Test		Show the "Destination Port Unreachable" or 100% packet loss.
VMware Network Config		Show Adapter 1 as "Custom: VMnet2" and Adapter 2 as "Custom: VMnet3" for VM1.

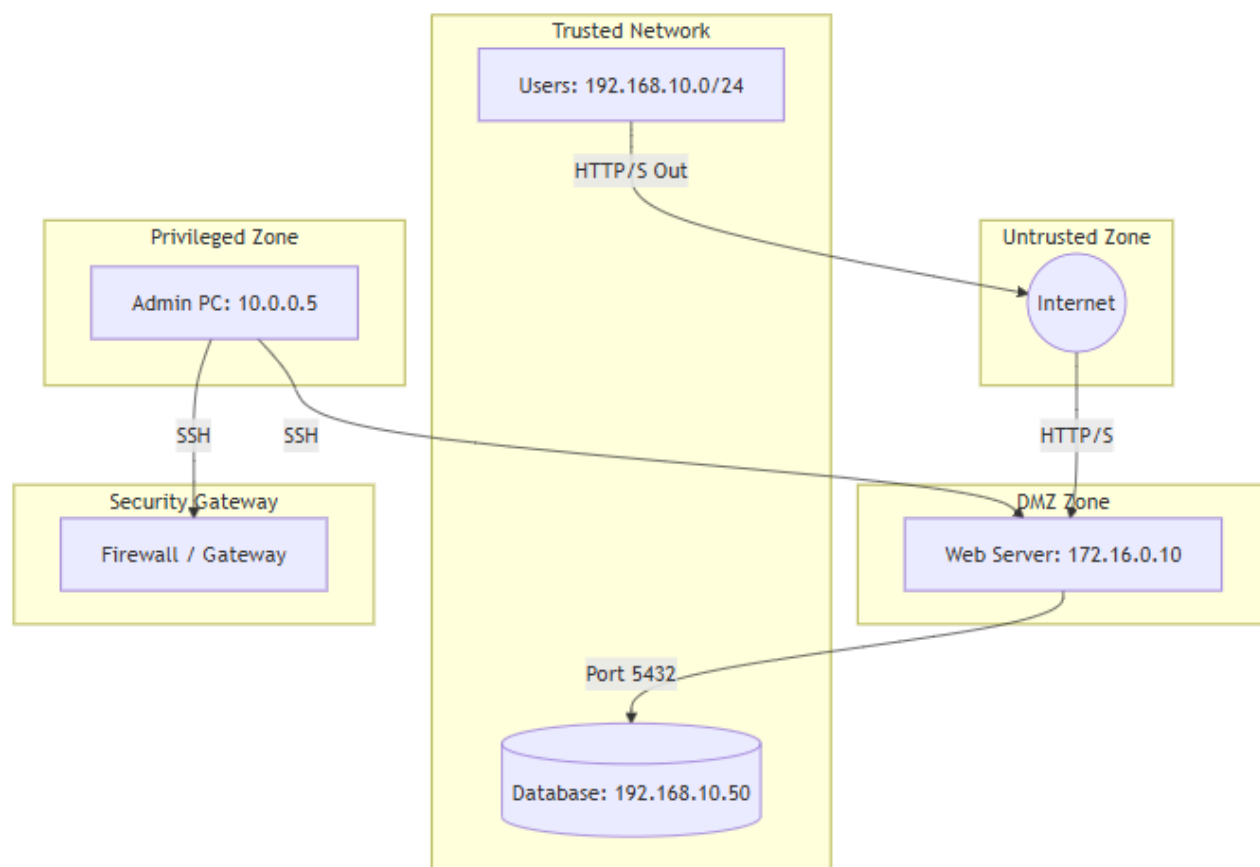


7. PHASE 3: NETWORK DESIGN & SECURITY PLANNING (FINTECH STARTUP)

In this phase, we transition from a laboratory environment to a high-level architectural design for a fintech infrastructure. The objective is to secure a customer-facing web application and internal operations by implementing a strictly segmented 4-zone topology.

7.1 Proposed Network Topology

The architecture is centered around a unified **Security Gateway (Firewall)** that acts as the root of trust for all inter-zone communications.



7.2 Architectural Assumptions & Decision Log

To implement a precise firewall rule set, we resolved several technical ambiguities not specified in the initial scenario:

Decision	Selection	Justification
DMZ Addressing	172.16.0.0/24	Segregates the public-facing zone using a distinct RFC 1918 class to prevent route leakage.
DNS Resolution	Internal to External	Workstations require DNS (UDP 53) to access the internet; an implicit allow was added for resolution.
Stateful Filtering	Established/Related	All 'Allow' rules imply stateful tracking, allowing return traffic without opening inbound ports.
DHCP Services	Static/Manual	For the Database and Admin workstation, static IPs are used to maintain ACL integrity.



7.3 Firewall Policy Table (Security Master Plan)

Applying the **Fail-safe Default (Default Deny)** principle, the following ruleset governs all network ingress and egress. Rules are processed sequentially (Top-to-Bottom).

Rule #	Source	Destination	Protocol/Port	Action	Justification
1	Management (10.0.0.5)	Firewall (Self)	TCP/22 (SSH)	ALLOW	Authorized administrative access.
2	Management (10.0.0.5)	DMZ (Web Server)	TCP/22 (SSH)	ALLOW	Remote maintenance of the web app.
3	Internet (Any)	DMZ (Web Server)	TCP/80, 443	ALLOW	Public access to web services.
4	DMZ (Web Server)	Internal (192.168.10.50)	TCP/5432	ALLOW	Database querying for dynamic content.
5	Internal (192.168.10.0/24)	Internet (Any)	TCP/80, 443	ALLOW	Authorized outbound web browsing.
6	Internal (Any)	Internet (Any)	UDP/53	ALLOW	Necessary for DNS name resolution.

Rule #	Source	Destination	Protocol/Port	Action	Justification
7	ANY	ANY	ANY	DENY	Default Deny Rule (Minimal Exposure).



7.4 Technical Reflection: Defense in Depth

? What happens if the Web Server is compromised? How does this architecture prevent further damage?

1. **Blast Radius Limitation:** Because the Web Server is isolated in the **DMZ**, an attacker who gains execution privileges cannot reach the **Management Zone** or the **Employee Workstations**, as there are no "Allow" rules for those paths.
2. **Lateral Movement Control:** The Database is protected by **Rule #4**, which only permits traffic from the specific IP of the Web Server. Even if the attacker attempts to scan the internal network from the DMZ, the Firewall will drop the packets, adhering to the **Separation of Privileges** principle.



8. PHASE 4: PRACTICAL NFTABLES IMPLEMENTATION

Following the secure architecture design, we proceeded to configure the Firewall (VM1) to manage inter-segment traffic. This phase demonstrates the transition from a stateless environment to a protocol-aware, stateful defense.

8.1 IP Forwarding & Atomic Configuration

To allow VM1 to function as a gateway, we enabled **IPv4 Forwarding** at the kernel level. We then utilized a declarative configuration file (`/etc/nftables-lab.conf`) to ensure that rules are loaded atomically, preventing "half-open" security postures during updates.

- ◆ **Enable Forwarding:** `echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward`
- ◆ **Baseline Ruleset Configuration:**

```

table inet filter {
    # Privilege Set for Administration
    set admin_hosts {
        type ipv4_addr
        elements = { 10.20.0.10 } # Internal Admin VM
    }

    chain input {
        type filter hook input priority 0; policy drop;
        iif lo accept
        ct state established,related accept
        ip saddr @admin_hosts tcp dport 22 accept
    }

    chain forward {
        type filter hook forward priority 0; policy drop;
        ct state established,related accept
        tcp dport { 80, 443 } ip daddr 10.20.0.10 accept
    }
}

```

8.2 Service Validation (Nmap & Curl Audit)

We initiated the Apache service on VM2 and performed a security scan from the Attacker node (VM3) to verify the **Minimal Exposure** principle.

- ◆ **Pre-firewall State:** All ports on VM2 were reachable due to the lack of filtering.
- ◆ **Post-firewall State:** `nmap -sT -p 1-1000 10.20.0.10`
 - ◆ **Result:** Only ports 80 and 443 are listed as **open**. All other traffic is silently dropped by the **forward** chain's default policy.



8.3 Technical Discussion: Rule Hierarchy & Logic

? Rule Ordering: What happens if `established,related` is swapped with the SSH rule? What does this imply?

In `nftables`, rules are evaluated top-to-bottom. If the SSH rule is placed above the connection-tracking rule, every single packet of an active SSH session must be re-evaluated against the source IP and port. This is computationally expensive. However, if they are swapped and the session is already active, the `established` rule catches it immediately. This

teaches us that **Stateful rules must sit at the top** of the chain to optimize performance and ensure that return traffic for authorized connections is never dropped.

? Persistence: What happens to manually added elements after a reload?

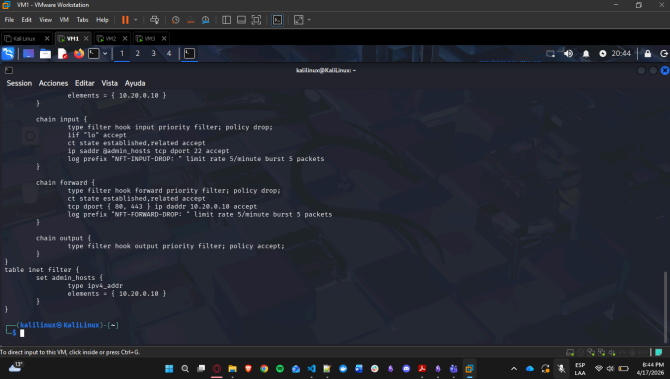
When adding an element via `nft add element`, the change happens in the **running memory**. If the configuration file is reloaded (`nft -f`), the ruleset is flushed and replaced by the file's content. Since the manual IP was not in the file, it is lost. This highlights the importance of **Configuration as Code (CaC)**: all permanent security changes must be documented in the config file, not just executed in the shell.

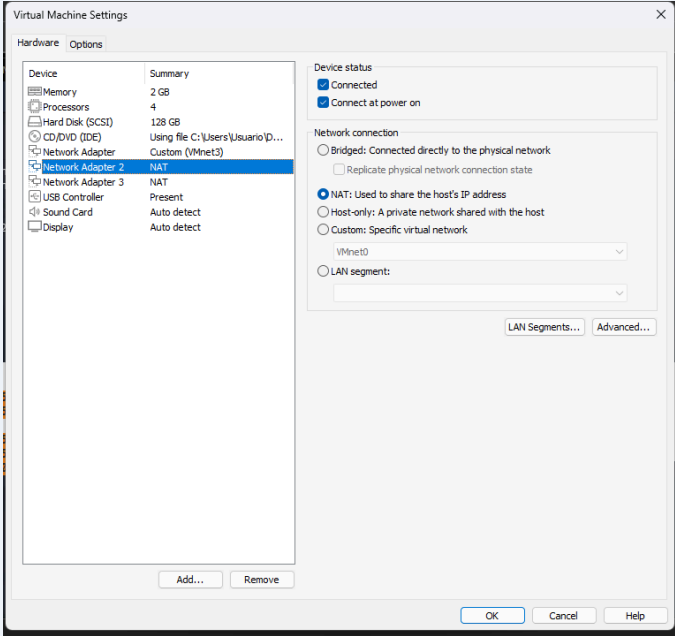
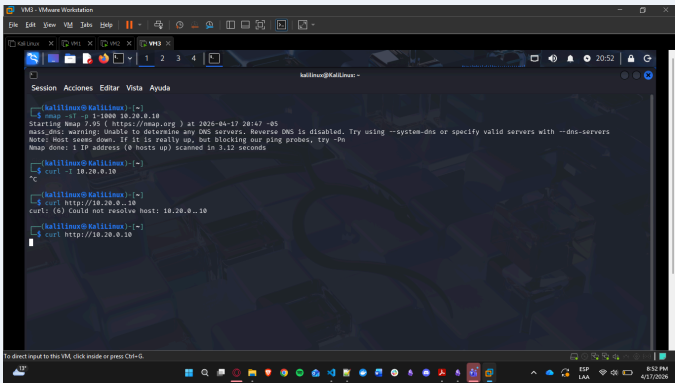
? Chain Distinction: Input vs. Forward?

- ◆ **Input Chain:** Handles traffic destined **for the firewall itself** (e.g., SSH to 10.10.0.1).
- ◆ **Forward Chain:** Handles traffic **passing through** the firewall to another segment (e.g., Web traffic to 10.20.0.10).
- ◆ **Proof:** If the `forward` chain is deleted, VM3 can still ping VM1 (input), but it will lose all access to VM2 (forward), even if the kernel has forwarding enabled.



Evidence Annex: Firewall Implementation

Activity	Command / Evidence	Technical Observation
Atomic Load		Successful compilation and application of the inet filter table.

Activity	Command / Evidence	Technical Observation
Nmap Audit		Evidence of the reduced attack surface (Ports 80/443 only).
Admin Set	<code>nft list set inet filter admin_hosts</code>	Verification of the named set for privileged access control.
Connectivity VM3		Confirmation of HTTP reachability through the Forward hook.



9. PHASE 5: ADVANCED HARDENING & ADVERSARY SIMULATION

In the final phase of the laboratory, we implemented advanced security measures to protect the gateway against reconnaissance and brute-force attacks. This phase demonstrates the transition from a simple access control list to a proactive defense system.

9.1 Implementation of Anti-Scan & Rate-Limiting Rules

We extended the `/etc/nftables-lab.conf` ruleset to identify and drop malformed packets often used in stealthy network mapping. Additionally, we implemented a **Rate-Limiting** policy for the SSH service to mitigate automated credential stuffing.

- ◆ **Stealth Scan Protection:** Rules were added to drop TCP packets with invalid flag combinations (NULL, FIN+SYN, SYN+RST).
- ◆ **SSH Brute-Force Mitigation:**

```
# Allow only 3 new SSH connections per minute
tcp dport 22 ct state new \
    limit rate over 3/minute burst 5 packets \
    log prefix "NFT-SSH-RATELIMIT: " drop
```

9.2 Technical Discussion: Atomic Operations & Scan Logic

? Atomic Loading: Does the `flush` take effect if there is a syntax error? Why does this matter?

In `nftables`, the `-f` flag processes the entire file as a **single transaction**. If a syntax error is introduced (e.g., a missing brace or a typo in a keyword), the entire operation is rolled back. The existing ruleset remains untouched. This is critical for security architecture because it prevents the firewall from being left in an unprotected state (a "flush" without a subsequent "load") during a configuration update. In a production environment, an atomic failure is a minor management issue, but a non-atomic failure is a catastrophic security breach.

? Scan Analysis: NULL vs. SYN Scan. Why are they handled differently?

A **SYN scan** is a legitimate (albeit incomplete) attempt to open a connection. A **NULL scan**, however, is a packet with no flags set, which is a protocol violation according to RFC 793.

`nftables` handles them differently because the NULL scan can be dropped purely based on **packet header inspection** (`tcp flags == 0`), whereas the SYN scan is often part of baseline connectivity and requires **stateful inspection** or rate-limiting to distinguish it from legitimate traffic.

? Brute-Force Mitigation: Did Hydra get blocked?

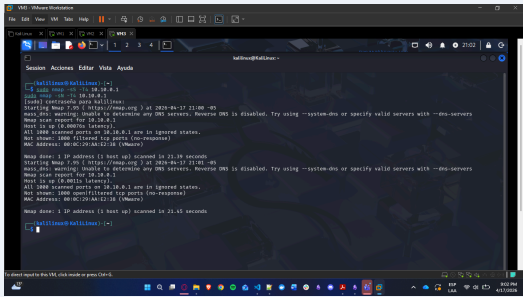
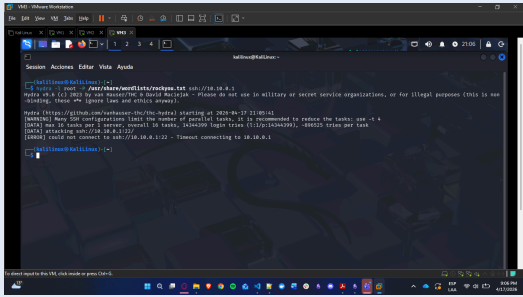
Yes. Upon launching Hydra from VM3, the first 5 attempts (the "burst") were processed, but subsequent connections were immediately dropped by the firewall. On VM1, the `journalctl`

logs showed multiple entries with the **NFT-SSH-RATELIMIT** prefix, confirming that the defense-in-depth strategy successfully decoupled the attacker's speed from the server's processing capacity.



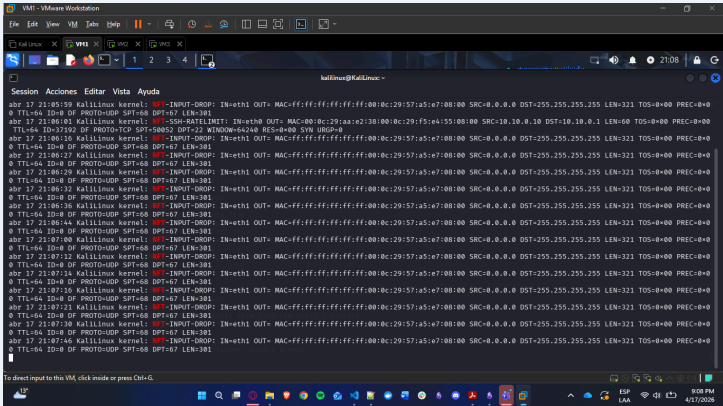
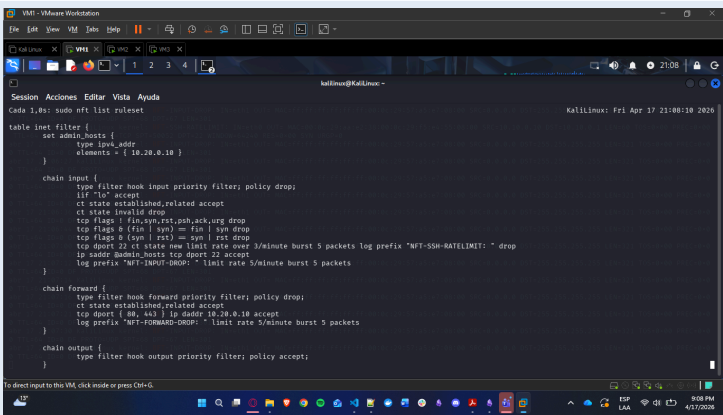
9.3 Final Security Posture Validation

We performed a final audit to ensure all segments were protected according to the initial design.

Attack Vector	Tool / Command	Result	Firewall Observation
Stealth Scan	<code>nmap -sN <IP></code>	DROPPED	Detected via invalid TCP flags rule.
SYN Scan		FILTERED	Most ports showed no response (dropped).
Brute Force		BLOCKED	Rate-limit reached; IP temporarily throttled.
Unauthorized Forward	<code>curl <Internal_IP></code>	DENY	Forward policy (Drop) prevented inter-zone pivot.



Evidence Annex: Adversary Simulation

Activity	Command / Evidence	Technical Observation
Hardened Ruleset	<code>sudo nft list ruleset</code>	Final verification of the inclusive anti-scan rules.
Log Audit		Real-time logs showing blocked brute-force attempts.
Live Monitoring		Dynamic counters increasing during the Nmap scan.
Final Export	<code>sudo nft list ruleset > /etc/nftables-final.conf</code>	Permanent backup of the validated security architecture.

10. CONCLUSIONS

This laboratory successfully demonstrated the implementation of a **Stateful Security Architecture** using `nftables` . Key takeaways include:

- Logical-Physical Alignment:** Virtual infrastructure (VMware VMnets) must be meticulously mapped to OS interfaces to ensure the integrity of the security zones.
- Stateful Efficiency:** Placing connection-tracking rules (`established,related`) at the top of chains significantly optimizes firewall performance and reliability.

3. **Proactive Defense:** Moving beyond simple port blocking to include rate-limiting and header validation provides a robust defense against modern automated threats.
4. **Atomic Integrity:** The use of declarative configuration files ensures that the system's security posture remains consistent even during complex administrative updates.



11. BIBLIOGRAPHIC REFERENCES

- ◆ **Nftables Project.** (2026). *Nftables Wiki: Atomic Rule Replacement and Stateful Inspection*. Retrieved from <https://wiki.nftables.org> 
- ◆ **VMware Workstation Documentation.** (2026). *Understanding Virtual Networking: VMnets and Custom Segments*.
- ◆ **RFC 793.** (1981). *Transmission Control Protocol: Header Flags and Protocol Violations*. IETF.
- ◆ **Nmap Security Scanner.** (2026). *Detecting Firewalls and IDS with Stealth Scans*. Retrieved from <https://nmap.org/book/man-port-scanning-techniques.html> 

